

EXPERIMENT 9

By Lakshya Yadav

Objective: Input two numbers and allow the user to swap them either using a third variable or without using a third variable. Display values before and after swapping.

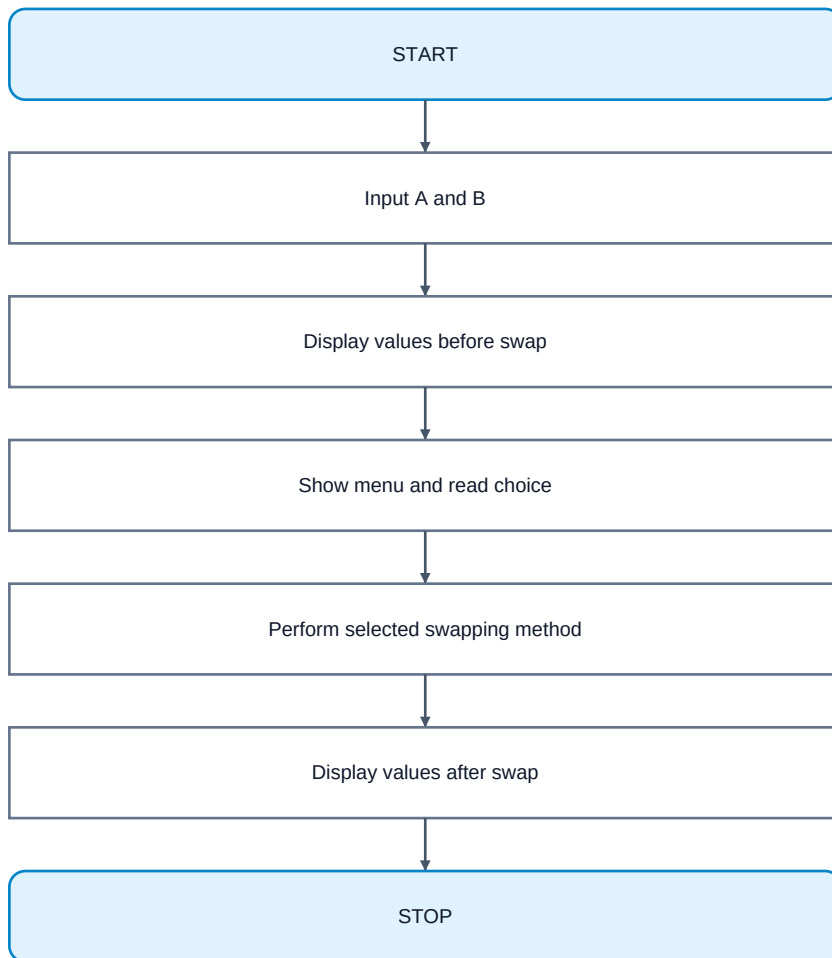
Algorithm

1. Start.
2. Input two integers A and B.
3. Display A and B before swapping.
4. Display menu: (1) swap using third variable, (2) swap without third variable, (3) exit.
5. Read choice.
6. For choice 1, use temp=A; A=B; B=temp.
7. For choice 2, use A=A+B; B=A-B; A=A-B.
8. Display values after swapping.
9. Stop.

Pseudocode

```
START
INPUT A, B
PRINT "Before swapping", A, B
PRINT menu
INPUT choice
IF choice = 1 THEN
    temp <- A
    A <- B
    B <- temp
ELSE IF choice = 2 THEN
    A <- A + B
    B <- A - B
    A <- A - B
ELSE
    EXIT
END IF
PRINT "After swapping", A, B
STOP
```

Flowchart



Test Case Table

Test	A	B	Choice	Expected Result
1	10	20	1	After swap: A=20, B=10
2	5	9	2	After swap: A=9, B=5
3	-3	7	1	After swap: A=7, B=-3

C Program

```

#include <stdio.h>

int main() {
    int a, b, temp, choice;

    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    printf("\nBefore swapping: a = %d, b = %d\n", a, b);
    printf("\n1. Swap using third variable");
    printf("\n2. Swap without third variable");
    printf("\n3. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            temp = a;
            a = b;
            b = temp;
    }
}

```

```
        break;
    case 2:
        a = a + b;
        b = a - b;
        a = a - b;
        break;
    case 3:
        return 0;
    default:
        printf("Invalid choice.\n");
        return 0;
}

printf("After swapping: a = %d, b = %d\n", a, b);
return 0;
}
```

Compile and Execute

```
gcc swap.c -o swap
./swap
```

Sample Run

```
Enter two integers: 10 20
Before swapping: a = 10, b = 20
1. Swap using third variable
2. Swap without third variable
3. Exit
Enter choice: 1
After swapping: a = 20, b = 10
```

2. Sum of Natural Numbers and Sum of Squares

Objective: Create a menu-driven program to calculate either the sum of the first n natural numbers or the sum of squares of the first n natural numbers.

Formulae

Sum: $S = n(n+1)/2$

Sum of squares: $S_2 = n(n+1)(2n+1)/6$

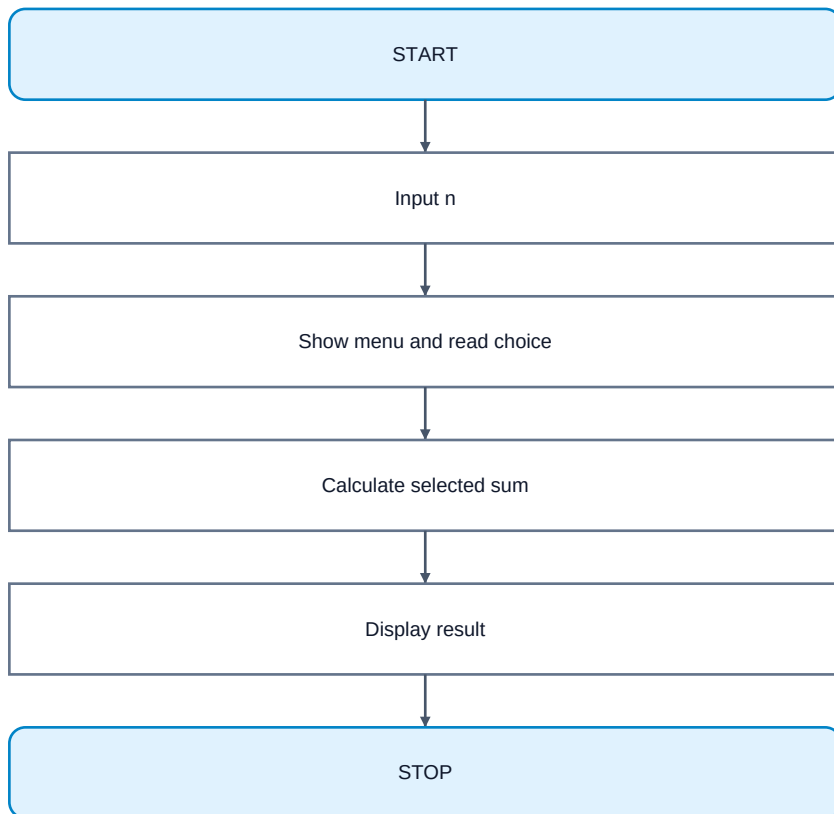
Algorithm

1. Start.
2. Input n.
3. Display menu: (1) sum of first n natural numbers, (2) sum of squares, (3) exit.
4. Read choice.
5. Apply the appropriate formula.
6. Display the result.
7. Stop.

Pseudocode

```
START
INPUT n
PRINT menu
INPUT choice
IF choice = 1 THEN
    sum <- n * (n + 1) / 2
    PRINT sum
ELSE IF choice = 2 THEN
    sumsq <- n * (n + 1) * (2*n + 1) / 6
    PRINT sumsq
ELSE
    EXIT
END IF
STOP
```

Flowchart



Test Case Table

Test	n	Choice	Expected Result
1	5	1	Sum = 15
2	5	2	Sum of squares = 55
3	10	1	Sum = 55
4	3	2	Sum of squares = 14

C Program

```

#include <stdio.h>

int main() {
    long long n, sum, sumsq;
    int choice;

    printf("Enter n: ");
    scanf("%lld", &n);

    if (n < 1) {
        printf("n must be a positive integer.\n");
        return 0;
    }

    printf("\n1. Sum of first n natural numbers");
    printf("\n2. Sum of squares of first n natural numbers");
    printf("\n3. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            sum = n * (n + 1) / 2;
            printf("Sum = %lld\n", sum);
    }
}

```

```
        break;
    case 2:
        sumsq = n * (n + 1) * (2 * n + 1) / 6;
        printf("Sum of squares = %lld\n", sumsq);
        break;
    case 3:
        return 0;
    default:
        printf("Invalid choice.\n");
    }
    return 0;
}
```

Compile and Execute

```
gcc sums.c -o sums
./sums
```

Sample Run

```
Enter n: 5
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
3. Exit
Enter choice: 2
Sum of squares = 55
```

3. Interest and Total Payable Calculator

Objective: Use principal (P), annual rate (R), and time in years (T) to calculate simple interest, compound amount, compound interest, and total payable.

Formulae

Simple Interest: $SI = P \times R \times T / 100$

Compound Amount (annual compounding): $A = P(1 + R/100)^T$

Compound Interest: $CI = A - P$

Total Payable: for simple interest, $P + SI$; for compound interest, A .

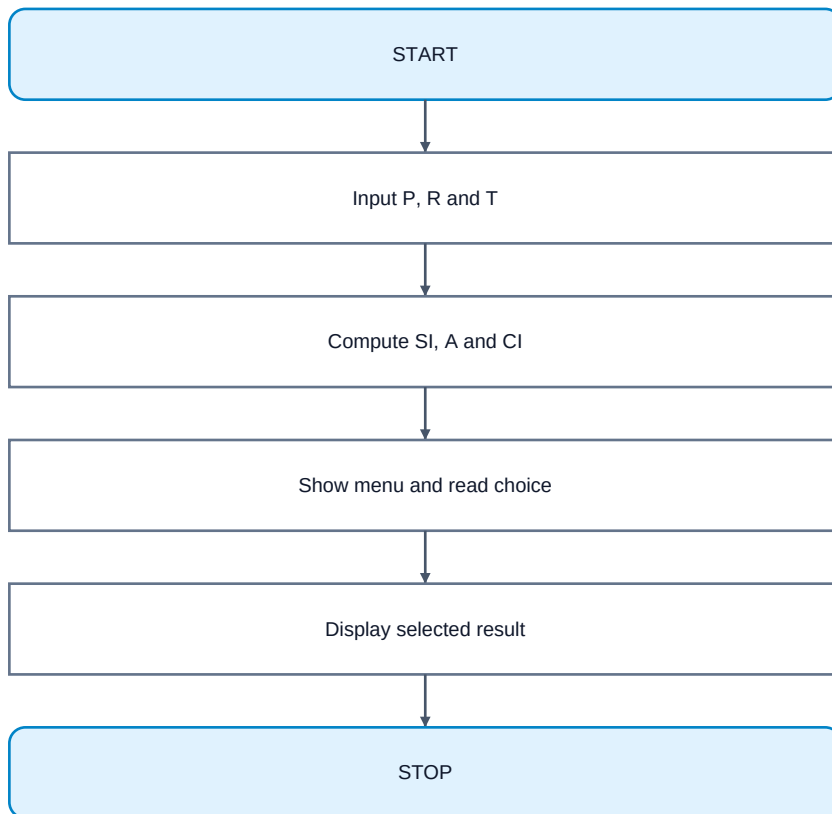
Algorithm

1. Start.
2. Input P, R and T.
3. Display menu for SI, compound amount, CI, total payable using SI, total payable using CI, and exit.
4. Read choice.
5. Compute the selected value using the formulae.
6. Display result.
7. Stop.

Pseudocode

```
START
INPUT P, R, T
SI <- P * R * T / 100
A <- P * (1 + R/100)^T
CI <- A - P
PRINT menu
INPUT choice
CASE choice OF
  1: PRINT SI
  2: PRINT A
  3: PRINT CI
  4: PRINT P + SI
  5: PRINT A
  6: EXIT
  OTHERWISE: PRINT "Invalid choice"
END CASE
STOP
```

Flowchart



Test Case Table

Test	P	R	T	Choice	Expected Result
1	1000	10	2	1	SI = 200.00
2	1000	10	2	2	A = 1210.00
3	1000	10	2	3	CI = 210.00
4	1000	10	2	4	Total (SI) = 1200.00
5	1000	10	2	5	Total (CI) = 1210.00

C Program

```

#include <stdio.h>
#include <math.h>

int main() {
    double p, r, t, si, amount, ci;
    int choice;

    printf("Enter principal, rate (%), and time (years): ");
    scanf("%lf %lf %lf", &p, &r, &t);

    si = (p * r * t) / 100.0;
    amount = p * pow(1.0 + r / 100.0, t);
    ci = amount - p;

    printf("\n1. Simple Interest");
    printf("\n2. Compound Amount");
    printf("\n3. Compound Interest");
    printf("\n4. Total Payable (Simple Interest)");
    printf("\n5. Total Payable (Compound Interest)");
    printf("\n6. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);
  
```

```

switch (choice) {
    case 1: printf("Simple Interest = %.2f\n", si); break;
    case 2: printf("Compound Amount = %.2f\n", amount); break;
    case 3: printf("Compound Interest = %.2f\n", ci); break;
    case 4: printf("Total Payable (SI) = %.2f\n", p + si); break;
    case 5: printf("Total Payable (CI) = %.2f\n", amount); break;
    case 6: return 0;
    default: printf("Invalid choice.\n");
}
return 0;
}

```

Compile and Execute

```

gcc interest.c -o interest -lm
./interest

```

Sample Run

```

Enter principal, rate (%), and time (years): 1000 10 2
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable (Simple Interest)
5. Total Payable (Compound Interest)
6. Exit
Enter choice: 3
Compound Interest = 210.00

```

Testing Conclusion

The programs produce the expected outputs for the prepared normal test cases. Invalid menu choices are also handled. For the natural-number program, n is checked to ensure that it is positive. The compound-interest program assumes annual compounding.

Submission Note

Save each program in a separate `.c` file, compile it with GCC using the commands shown above, execute it, and compare the actual output with the expected values in the test-case tables.